

Mars Rover Navigation Group 2

F20RO/F21RO Coursework

Abstract

Group 2 coursework using Behaviour-Based Robotics and Evolutionary Robotics to navigate a rover through a Mars environment.

Contents

Mars Rover Navigation Group 2	0
1. Introduction	2
2. Implementation	3
2.1 Behaviour-Based Robotics (BBR)	3
2.2 Evolutionary Robotics (ER)	4
2.2.1. Overview	4
2.2.2. Multilayer Perceptron (MLP)	5
2.2.3. Fitness Function	6
2.2.4. Genetic Algorithm	7
2.2.5. Summary of Parameters	7
3. Results and Analysis	8
3.1 Results overview	8
3.2 Behaviour-Based Robotics (BBR) Results	8
3.3 Evolutionary Robotics (ER) Results	9
4. Discussion and Conclusion	11
5. Bibliography	13

1. Introduction

As a group of three we have been given a task to create a mars rover navigation system to help increase our understanding of robotic simulators as well as the software tools used to control them. Our goal was to create intelligent robot controllers using behavioural and evolutionary robotics approaches. The intelligent robots should be able to follow a set path towards an end goal to collect soil samples on mars with speed and obstacle avoidance being the two most important requirements. In addition to this, our robot should be able to exhibit decision making ability when selecting a correct route to its end goal. The correct route will depend upon the status of a light beacon positioned in the robot's environment. The robot should take the left path if the light beacon is active, and the right path if the beacon is inactive.

This report analyses the performance of behavioural and evolutionary robots designed to perform effectively in their environment. It will discuss the approach taken for each robot as well as the advantages and disadvantages of the different approaches. An example of the environment used for this experiment can be viewed in figure 1.

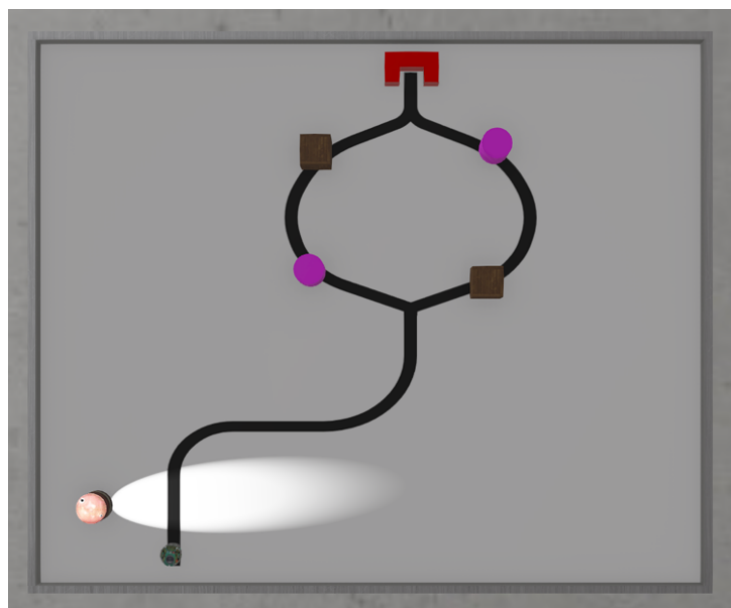


Figure 1. Simulator Environment

2. Implementation

2.1 Behaviour-Based Robotics (BBR)

The Mars rover navigation system implemented in “Webots” exhibits a meticulous design and execution, emphasizing the intricacies of line following, decision-making, and obstacle avoidance. The basic code for these mechanisms was inspired by the material provided in lab 1 - “e-puck_line_lab1.py”, and lab 2 - “e-puck_light_lab2.py”. Obviously, these had to be adapted to fit the requirements of this assignment.

At the core of the rover's navigation ability is a complex line-following mechanism. Ground sensors continuously provide real-time feedback on the rover's position relative to the desired trajectory. The rover dynamically adjusts its velocity based on the inputs from these ground sensors, ensuring precise tracking of the intended path. This intricate exchange between sensor feedback and motor control enables the rover to navigate along the designated route with precision. The line-following algorithm is designed to respond dynamically to variations in sensor inputs. It analyses the different sensor readings, adjusting the rover's velocity accordingly. For instance, when one side's ground sensor detects a stronger signal than the others, the rover adjusts its trajectory to maintain alignment with the desired path.

Complementing the rover's ability to follow the designated path is its decision-making process based on the status of a simulated surface beacon (represented by light sensors). This mechanism adds a layer of adaptability to the rover's navigation. If the beacon is activated, indicating lights are turned on, the rover follows Route A (left). Conversely, if the beacon is deactivated, signaling lights turned off, the rover takes Route B (right). This process introduces a dynamic element to the navigation system, allowing the rover to respond to changing environmental conditions. Its integration enhances the rover's autonomy and adaptability during its mission.

Navigating this map presents the rover with the constant threat of obstacles. The obstacle avoidance mechanism utilizes front-facing proximity sensors to detect impediments in its path. Upon detecting an obstacle, the rover initiates a calculated maneuver to circumvent the obstacle on the right, keeping its proximity within a specified window, using the left side distance sensors. A critical element of the obstacle avoidance strategy is the restriction on the number of avoidance maneuvers to twice. While this approach is viable when the number of obstacles is known, it

poses challenges in scenarios with an unknown number of obstacles. Attempts were made to address this by using only distance sensors. However, the imperfect nature of these sensors' readings led to complications, causing the rover to incorrectly treat the collection point as an obstacle. This limitation underscores the complexity of adapting obstacle avoidance strategies to dynamic and uncertain environments. This balancing act between swift navigation and ensuring the rover's safety is achieved through careful experimentation with different values of motor speed and distance thresholds to the obstacle.

The Mars rover confirms its arrival at the collection point by monitoring the readings from its four front-facing proximity sensors. When all these sensors detect values surpassing a predefined threshold, the rover interprets this as proximity to a 'U-shaped' obstacle. In response, it ceases movement by stopping both wheel motors. Additionally, a concise message is printed to signify the rover's successful arrival at the designated collection point.

The code is structured with a focus on modularity, encapsulating specific functionalities within dedicated methods. Each method corresponds to a distinct aspect of the rover's behavior, such as obstacle avoidance, line following, and decision-making. This modular design not only enhances code readability but also facilitates future modifications and additions to the system.

In conclusion, the design and implementation of the Mars rover navigation system reflects a nuanced consideration of the challenges present in the problem posed. The intricate navigation mechanisms collectively contribute to a system capable of handling the complexities of this simplified Mars terrain. The iterative experimentation and balancing of parameters demonstrate the attempt to achieve an optimal balance between efficiency and reliability in the rover's mission.

2.2 Evolutionary Robotics (ER)

2.2.1. Overview

Like evolution in living organisms, it can take many generations for a robot to develop some of the complex functionality that will enable it to thrive in its given environment. The robot in this experiment was gradually evolved over many

generations where each generation consisted of a small population of robots with different characteristics. Each robot would be tested in the environment multiple times and exposed to both scenarios where the light was on or off. If the robot took the correct path based on the light, it would receive an appropriate reward to encourage the correct decision.

2.2.2. Multilayer Perceptron (MLP)

Our implementation of our evolutionary robot is based on a multilayer perceptron (MLP) which is a type of neural network. An MLP takes in inputs, processes these through a series of interconnected layers and then outputs a final result. MLPs are extremely versatile and can be used in several different applications where approximations and predictions on unseen data are needed. In our case, we would feed the MLP the various inputs from our robot which consisted of three infrared (IR) sensors, eight proximity sensors and one light sensor. The MLP would then perform various operations based on this input data and it would output two values that were used to set the left and right motor speeds for our robot. This allowed our robot to navigate its environment based on a predicted appropriate action calculated from its received inputs.

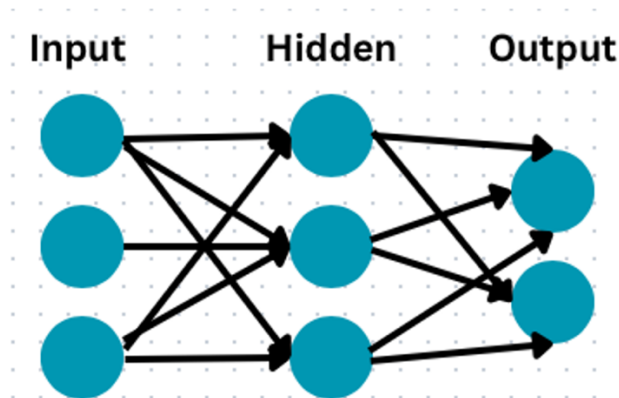


Figure 2. Basic MLP structure.

MLPs use an activation function to avoid linear behaviour and encourage it to spot complex patterns in the data. In our MLP, we used a sigmoid activation function which normalises the values between 0 and 1. These values can be thought of as probabilities where the closer a value is to 1, the more likely the associated event will occur. The MLP plays a key role in the adaptation of our robot.

2.2.3. Fitness Function

One of the most crucial aspects involved in effectively evolving robots is the use of an appropriate fitness function. These are used to evaluate the performance of a robot against how well it performed its intended goal within its environment. A robot with a strong evaluated fitness will stand a much greater chance of having its genes passed on to future generations.

For this task, a number of different fitness functions were trialled with each showing a varying degree of success. The first fitness function used was based on the one developed by Floreano and Mondada for their obstacle avoiding robot [1] (Floreano, et al., 1994). This fitness function was further modified to incorporate line-following behaviour and can be viewed in equation 1.

$$f = V(1 - \sqrt{\Delta v}) * (1 - i) * (1 - l) \quad (1)$$

where V is the average rotation speed of both wheels, Δv is the absolute difference in speed between these wheels, i is the value of the highest activated proximity sensor and l is the mean value of the IR sensors. It is important to note that the lower the l value is the closer the robot will be to the line.

The second fitness function tested was based on a weighted sum of various measurements taken from our robot. The measurements chosen reflected some of the desired behaviours of our robot which could be weighted to encourage the robot to favour specific actions over others. This can be viewed in equation 2.

$$f = xV + yl - zi \quad (2)$$

where V is the average rotation speed of both wheels, l is the mean value of the IR sensors, i is the value of the highest activated proximity sensor and x , y and z are constants.

The third fitness function tested used a combination of two equations conditioned on an input from our robot. If a robot detected an obstacle within its proximity, then equation 3 would be applied. If no obstacle was detected, then the robot's fitness would be calculated using equation 4.

$$f = V(\sqrt{\Delta v}) * (1 - i) - l \quad (3)$$

$$f = V(1 - \sqrt{\Delta v}) * (1 - i) + l \quad (4)$$

where V , Δv and i are the same as previously outlined. However, this time l was calculated by adding 0.33 for each IR sensor that was positioned on the line. In the case where all three of the robot's IR sensors were on the line, l would be equal to 0.99.

2.2.4. Genetic Algorithm

To implement the evolutionary techniques required to evolve our robot, a genetic algorithm was used. This algorithm was responsible for randomly mutating and crossing-over the genes from our robot population. We could customise the rate at which these operations were applied by modifying the mutation percentage and crossover rate respectively. By setting a higher mutation rate, we encourage more new behaviours to present themselves which may be useful for solving complex problems within an environment. We could also modify the crossover rate to determine how the genes of two parents will be combined when creating successive generations (Jong Choi, et al., 2018).

2.2.5. Summary of Parameters

Due to the complexity of evolutionary robotics, there are several parameters that can be modified to alter the evolutionary process. A summary of the parameters used when evolving our strongest performing robots can be viewed in table 1.

Section	Parameter	Value
Evolution	Generations	200
	Population	5
	Elite	1
MLP	Input Layers	11
	Hidden Layers	7
	Output Layers	2
Genetic Algorithm	Crossover rate	70
	Mutation Percentage	10

Table 1. Parameter summary.

3. Results and Analysis

3.1 Results overview

A summary of the times taken for the behavioural and evolutionary robots to complete their required tasks can be viewed in table 2.

	Time in Minutes			
	Run 1	Run 2	Run 3	Average
BBR Route A	4:39	4:39	4:39	4:39
BBR Route B	4:49	4:49	4:49	4:49
ER Route A	1:59	2:03	2:01	2:01
ER Route B	2:00	2:01	2:05	2:02

Table 2. Measured time taken for each task.

3.2 Behaviour-Based Robotics (BBR) Results

As Expected, when it came to our behaviour-based robot it would always take the same path no matter how many runs of the simulation we would do due to the environment being the same and as well as all objects in the environment being stationary. This is very clear in our results as they have the same time take to travel in each attempt though there was a difference in time between the two paths with

route A being 10 seconds faster than route B. The reasons for this time difference could be due to two factors or a combination of both that being that one route is shorter than the other or the way that our robot behaviours is set up reacted better and more optimally for route A due to the change of the obstacles order or position.

3.3 Evolutionary Robotics (ER) Results

The success of our robot evolution varied significantly depending on the fitness function used. However, we found that when using each of the previously mentioned fitness functions our robot was able to develop some of its basic functionality quickly. For example, we could observe our robot beginning to follow the line whilst travelling forward at a reasonable speed in early generations, usually around generation 25.

We found that the best results achieved used a combination of fitness functions 3 and 4. These fitness functions allowed for quick evolution in early stages and a steady increase in performance thereafter. The combination of these fitness functions helped the robot perform well in more complex situations like obstacle avoidance. The first fitness function enforced fast speed and line following but it did not fully address the issue of obstacle avoidance. This issue was mainly addressed using the second function where the robot's performance rating would decrease when it was positioned on the line and an obstacle was detected. This allowed the robots to quickly learn to avoid obstacles whilst also maintaining their line following ability, two skills that conflicted each other when using other fitness functions. The fitness achieved using this method at each generation can be viewed in figure 3.

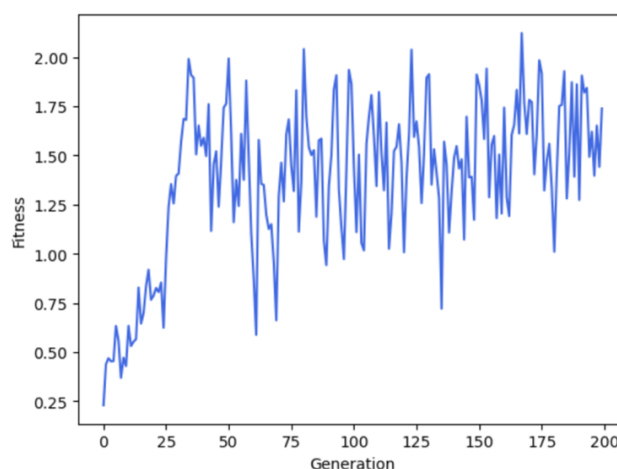


Figure 3. Average fitness at each generation.

We can see from this graph that our robot fitness increases significantly at generation 25, at this stage it was consistently following the line and achieving its first correct route reward. After this we can observe a gradual upwards trend where the robot makes small and steady improvements as the generations increase. In this experiment, our robot was making it to the end goal by generation 124.

During our experimentation, we noticed several trends across the evolutionary processes. One interesting trend observed, was the importance of the initial generation. In our experiment, our initial population was generated randomly which inevitably meant that some initial populations contained more capable robots than others. As expected, the experiments that began with stronger robots would begin to evolve faster due to the presence of stronger genes within the population. Unfortunately, this initial advantage had a big effect on the evolution of robots throughout the entire experiment and not just the early stages. This meant that experiments that began with robots with poorer genes would not evolve nearly as fast as those with good initial genes. In some cases, the impact of poor initial genes was so significant that the robots would never develop some of the more complex functionalities required to thrive in their environment, mainly due to the robots getting stuck in a local optima. This issue could be addressed by employing a method to generate an initial population containing good genes which would help increase the rate of evolution and avoid any plateaus in the evolutionary process. One method to achieve this could be by using heuristic methods to consistently generate a more capable initial population [3].

Another interesting behaviour noticed was how significantly the speed of the robot could impact its learning ability. We observed that robots travelling at a fast speed struggled to develop more complex functionality such as obstacle avoidance. This may have been due to the robot having less time to explore and analyse some areas of its environment which may have led to it missing learning opportunities. It also could be the case that our robot was receiving too high a reward for its fast speed, leading it to neglect other reward opportunities. To combat this issue, we could modify the fitness functions to depend less on speed or increase the duration of our simulation.

4. Discussion and Conclusion

In our pursuit to develop a Mars rover navigation system our team effort has given each of us an insight into both Behaviour-Based Robotics (BBR) and Evolutionary Robotics (ER) approaches. With each method having its own set of challenges. When it came to behaviour-based robotics we had to get it to react to its environment based on the information it was getting from its sensors and make decisions accordingly. Therefore, we coded the robot to follow lines, avoid obstacles and sense light to pick a path and because of that the movement it would make was very predictable and consistent with that showing in the results for the Behaviour-Based robot with it taking the same time for each time it went the same path. This shows the robustness of Behaviour-Based Robotics in our static environment which was the same conclusion made in a research paper using Line following and Obstacle avoidance with them saying “The simulation and real time experiments validate the effectiveness and the robustness of the proposed method in static and dynamic environments.” [4]. When we compared the Behaviour-Based Robotics to Evolutionary Robotics it was almost the opposite with the fact that it was very unpredictable and inconstant with times where the robot would not reach the goal if the robot got stuck in a local optima. This is because we are reliant on the robot teaching itself how to get to the goal rather than following instructions based upon its environment with the only influences we have over it being the fitness function as well as rewards to point it in the right direction and get behaviours we want it to have such as line following and obstacle avoidance using fitness function and giving it points if it reached specific checkpoints on the path to the goal. In the end we did get faster times for the Evolutionary Robotics once it had learnt how to get to the goal and optimise its route to get there the fastest with each iteration of new generations leading us to get times between 1:59- 2:05 allowing us to draw the conclusion that despite Evolutionary Robotics initial unpredictability it caused it to be great for optimizing the Mars rover navigation system. With the contrasting natures of the two navigation systems highlight the trade-off of each system methodology.

In conclusion developing the mars rover navigation system gave us an in-depth look at the contrasting robotic behaviours of the two implemented navigation system. While Behaviour-Based Robotics provides reliability and predictability Evolutionary Robotics offers adaptability and learning potential. With the way that Behaviour

based robotics is implemented by coding the robot to move via the sensor data of the robot and Evolutionary robotics being implemented by fitness factors with a point system. The fusion of insight from both approaches could pave a way to a hybrid navigation system that combines the strengths of deterministic responses with the flexibility of evolutionary learning. As we reflect on what we learnt throughout the coursework it will inspire us to think of future usages and innovation of advancements in intelligent robot controllers.

5. Bibliography

- [1] Floreano, D., Mondada, F., Cliff, D. & Husbands, P., 1994. *Automatic Creation of an Autonomous Agent: Genetic Evolution of a Neural Network Driven Robot*. s.l.:s.n.
- [2] Jong Choi, T., Yun-Gyung, C. & Chan Wook, A., 2018. *A Performance Comparison of Crossover Variations in Differential Evolution for Training Multi-layer Perceptron Neural Networks*. West Region: Springer, Singapore.
- [3] Mohammadi, . A. & Khaji, E., 2014. *A Heuristic Method to Generate Better Initial Population for Evolutionary Methods*. Gotenburg: School of Applied Physics, University of Gothenburg.
- [4] Nicolescu, F. M. & M., 2016. *SpringerLink*. [Online]
Available at: <https://core.ac.uk/download/pdf/52956867.pdf>